

Scripts PowerShell

Utilisation de Windows PowerShell ISE



OBJECTIFS

- Concevoir des scripts shell
- Importance et utilité des fonctions

Rester dans votre répertoire personnel P:\Documents (utilisez la commande set-location lequel vous allez éditer et tester vos scripts shell.

RAPPELS

Une fonction est un bout de code, un ensemble de commandes et d'instructions qui effectuent une tâche spécifique.

Une fois qu'une fonction est définie, elle peut être appelée ou utilisée plusieurs fois dans le script. Une fonction va nous permettre d'éviter la redondance du code et rend le script plus propre et plus facile à comprendre.

Si vous avez besoin d'effectuer 5 fois, 10 fois ou 20 fois la même action dans un script, vous pouvez créer une fonction pour effectuer l'action désirée. Ainsi, à chaque fois que vous avez besoin d'exécuter cette action, il vous suffit d'appeler la fonction par son nom. Ceci évite de dupliquer X fois le même bout de code. De plus, si vous devez modifier ce bout de code pour corriger un bug, ajouter une évolution, etc... Il vous suffit d'adapter la fonction, alors que si vous dupliquez un bloc de code, vous devrez modifier X bloc de code.

Le code d'une fonction est réutilisable à souhait : dans un même script, ou dans un autre script. Ceci va permettre d'avoir des scripts mieux organisés, plus facilement lisible et maintenable. Pour autant, l'utilisation d'une fonction ne doit pas être systématique, notamment si le code est unique et n'a pas vocation à être réutilisé.

Les fonctions peuvent également aider à isoler les variables. En effet, lorsqu'une variable est définie à l'intérieur d'une fonction, elle n'existe que dans le contexte de cette fonction. Cela signifie que vous pouvez utiliser le même nom de variable dans différentes fonctions sans craindre de conflits.

SYNTAXE D'UNE FONCTION

En PowerShell, sachez qu'il est possible d'écrire des fonctions sans paramètres et des fonctions avec paramètres. Tout dépend de nos besoins. Nous allons étudier les deux syntaxes.

Fonction simple sans paramètre

La syntaxe de base pour définir une fonction est la suivante :



```
function NomDeLaFonction {  
    # Instructions / code à exécuter lors de l'appel de cette fonction  
}
```

Pour le nom de la fonction, nous essaierons de respecter les bonnes pratiques de Microsoft, à savoir utiliser un verbe approuvé (Get, New, Set, etc...) suivi d'un tiret puis d'un nom.

Ainsi, nous pourrions écrire la fonction "Write-Hello" qui a un seul objectif : retourner "Bonjour !" dans la console, à chaque fois qu'elle est appelée, voici le code de la fonction :

```
function Write-Hello {  
    Write-Output "Bonjour !"  
}
```

Dans le script, pour appeler cette fonction, il suffira d'écrire ceci :

```
Write-Hello
```

Ce qui signifie que si vous écrivons 3 fois "Write-Hello" dans notre script, nous allons appeler 3 fois notre fonction. Voici un exemple :

```
PS C:\Users\Florian> function Write-Hello {  
    Write-Output "Bonjour !"  
}  
  
Write-Hello  
Write-Hello  
Write-Hello  
Bonjour !  
Bonjour !  
Bonjour !
```

Note : dans votre code, la fonction doit toujours être déclarée avant qu'elle ne soit appelée.

Fonction avec des paramètres

La fonction Write-Hello que nous venons d'écrire est statique puisqu'elle renverra toujours le même résultat. Nous ne pouvons pas jouer sur son comportement ou sur le résultat retourné, car elle n'a pas de paramètres. Au même titre qu'un cmdlet PowerShell, une fonction PowerShell peut avoir un ou plusieurs paramètres.

Dans une fonction, un bloc "**param()**" doit être intégré pour déclarer des paramètres entre les parenthèses.



```
function Write-Hello {  
    param($Parametre1,  
        $Parametre2  
    )  
    Write-Output "Bonjour !"  
}
```

Nous verrons par la suite que le paramètre peut avoir une valeur par défaut, ainsi qu'une configuration (ou "des paramètres de paramètres"). Ceci peut être utile pour le rendre obligatoire.

Nous allons faire évoluer notre exemple précédent. Nous pourrions ajouter le paramètre "-Language" permettant de préciser la langue (sur deux lettres) dans laquelle nous souhaitons dire bonjour. Ensuite, dans la fonction, nous devons analyser la valeur associée à ce paramètre pour retourner bonjour dans la langue sélectionnée. Nous ferons en sorte que, par défaut, la langue soit le français.

Voici le code de cette fonction au sein de laquelle nous utilisons un switch pour gérer les différentes langues :

```
function Write-Hello {  
  
    param($Language = "FR")  
  
    Switch($Language)  
    {  
        "FR" { $Hello = "Bonjour" }  
        "EN" { $Hello = "Hello" }  
        "ES" { $Hello = "Hola" }  
        "IT" { $Hello = "Buongiorno" }  
        default { $Hello = "Bonjour" }  
    }  
  
    return $Hello  
}
```

Désormais, nous pouvons appeler notre fonction, et selon le langage précisé, une valeur différente sera retournée. Si nous ne précisons aucune valeur, le "FR" sera utilisé par défaut puisque nous avons précisé "= "FR"" à la suite du nom du paramètre. Si nous précisons une valeur non prise en charge par la fonction, par exemple "CN", nous aurons quand même un résultat qui sera celui précisé dans l'instruction "default" du Switch (dans d'autres cas, ceci pourrait générer une erreur d'exécution de la fonction).



Nous devons indiquer **"-Language"** suivi d'une valeur, car ce paramètre est explicitement nommé "Language" au moment de sa déclaration dans la fonction.

```
Write-Hello
Write-Hello -Language "ES"
Write-Hello -Language "EN"
```

```
Write-Hello
Write-Hello -Language "ES"
Write-Hello -Language "EN"
Bonjour
Hola
Hello
```

Voici le résultat :

Vous devez savoir que l'instruction **"return"** permet d'indiquer ce que la fonction doit retourner. Ici, il s'agit de retourner la valeur de la variable "\$Hello", mais ça pourrait être un tableau, un objet, etc. Par ailleurs, sachez que la **portée de la variable "\$Language" est limitée** puisqu'elle existe uniquement au sein de la fonction.

Typing un paramètre de fonction

Enfin, pour finir, nous allons voir comment "typer" un paramètre dans une fonction. Ceci signifie que nous allons imposer le type d'une variable, sans laisser la main à PowerShell.

Que ce soit un paramètre de fonction ou une variable dans un script, la syntaxe est la même : le type doit être précisé entre crochets, avant le nom de la variable. Si nous souhaitons utiliser le type **"String"** pour le paramètre **"-Language"** puisque nous attendons une chaîne de caractères, nous allons utiliser ceci :

```
function Write-Hello {

    param([string]$Language = "FR")

    Switch($Language)
    {
        "FR" { $Hello = "Bonjour" }
        "EN" { $Hello = "Hello" }
        "ES" { $Hello = "Hola" }
        "IT" { $Hello = "Buongiorno" }
        default { $Hello = "Bonjour" }
    }

    return $Hello
}
```



CONTROLES SUR LES PARAMETRES DES FONCTIONS

Lors de la création d'une fonction en PowerShell, nous avons l'opportunité d'ajouter un ensemble de paramètres pour cette fonction. Désormais, nous allons apprendre à ajouter des contrôles sur ces paramètres de fonction, ce qui permettra de vérifier les données envoyées en entrée de la fonction. Ceci permettra également de rendre un paramètre obligatoire ou non.

Rendre un paramètre obligatoire avec mandatory

Pour certaines de vos fonctions, vous aurez certainement besoin de rendre **un ou plusieurs paramètres obligatoires**. En effet, si vous attendez une valeur sur un paramètre et que l'utilisateur n'indique pas de valeur, ceci pourra "faire planter" la fonction et retourner un résultat inattendu.

Nous allons prendre pour exemple la fonction nommée "**Get-Login**". Cette option permet de **retourner** un identifiant sous la forme "<Première lettre du prénom>.<nom>" à partir d'un prénom et d'un nom envoyé à la fonction. Pour cela, nous avons deux paramètres : "**\$Nom**" et "**\$Prenom**". Cet identifiant doit être retourné en minuscule.

```
function Get-Login {  
  
    param($Nom,  
        $Prenom)  
  
    $Identifiant = (($Prenom).Substring(0,1)).ToLower() + "." + ($Nom).ToLower()  
  
    return $Identifiant  
}  
  
$Login = Get-Login -Prenom "Jacques" -Nom "OUZI"
```

Cette fonction PowerShell retourne bien le résultat attendu puisqu'avec le prénom "Jacques" et le nom "OUZI", nous obtenons l'identifiant "f.ouzi".

Le problème, c'est qu'actuellement, je peux appeler la fonction en omettant l'une des deux valeurs, ou les deux valeurs. Ceci va retourner une erreur.

```
PS C:\Users\Utilisateur> $Login = Get-Login -Prenom "Jacques"  
Impossible d'appeler une méthode dans une expression Null.  
Au caractère 0: _DATA_SIO_BLOC_1\PowerShell\script\Get-Login.ps1:6 : 5  
+ ~~~~~  
+ $Identifiant = (($Prenom).Substring(0,1)).ToLower() + "." + ($Nom ...  
+ ~~~~~  
+ CategoryInfo          : InvalidOperation : (:) [], RuntimeException  
+ FullyQualifiedErrorId : InvokeMethodOnNull  
  
PS C:\Users\Utilisateur>
```

Nous allons modifier la fonction pour rendre les deux paramètres obligatoires. Ainsi, la fonction refusera de s'exécuter s'il manque le prénom et/ou le nom. Nous devons ajouter l'instruction

"**[Parameter(Mandatory=\$true)]**" devant le nom de chaque paramètre à rendre obligatoire. Enfin, c'est surtout le positionnement du paramètre "**Mandatory**" sur "**\$true**" qui le rend obligatoire.

```
function Get-Login {  
  
    param([Parameter(Mandatory=$true)]$Nom,  
          [Parameter(Mandatory=$true)]$Prenom)  
  
    $Identifiant = (($Prenom).Substring(0,1)).ToLower() + "." + ($Nom).ToLower()  
  
    return $Identifiant  
}
```

Désormais, si nous appelons la fonction sans préciser le prénom ou le nom, PowerShell restera en attente d'une valeur.

```
$Login = Get-Login -Prenom "Jacques"
```

Voici un exemple où la fonction est appelée sans valeur pour le paramètre "-Nom" :

```
PS C:\Users\Utilisateur> $Login = Get-Login -Prenom "Jacques"  
applet de commande Get-Login à la position 1 du pipeline de la commande  
Fournissez des valeurs pour les paramètres suivants :  
Nom :
```

Si nous indiquons une valeur et que nous appuyons sur Entrée, la fonction sera exécutée. Cet exemple montre bien l'importance et l'utilité de pouvoir rendre obligatoire certains paramètres de fonction.

Contrôle sur les paramètres

En plus de jouer sur le caractère obligatoire ou non d'un paramètre, vous pouvez ajouter des contrôles sur les paramètres. Ceci est très pratique pour s'assurer que la **valeur correspond à un format précis**, que la **valeur correspond à un ensemble de valeurs acceptées**, que la **valeur respecte une certaine longueur** ou encore pour **exécuter une commande de validation de cette valeur**.

ValidatePattern

Pour commencer, nous allons peaufiner notre exemple précédent en ajoutant un contrôle sur les paramètres "\$Nom" et "\$Prenom" afin de **nous assurer qu'ils ne contiennent que des lettres, en minuscules ou majuscules**. Ceci évite qu'un chiffre ou un nombre soit indiqué dans le prénom et/ou le nom.

Nous devons ajouter le paramètre "**ValidatePattern**" qui permet de définir une expression régulière pour valider la valeur associée au paramètre en question. Ici, nous vérifions l'utilisation de lettres, exclusivement.



```
function Get-Login {

    param([Parameter(Mandatory=$true)][ValidatePattern("^[a-z]+$")]$Nom,
          [Parameter(Mandatory=$true)][ValidatePattern("^[a-z]+$")]$Prenom)

    $Identifiant = (($Prenom).Substring(0,1)).ToLower() + "." + ($Nom).ToLower()

    return $Identifiant
}
```

Ainsi, si nous exécutons ceci :

```
$Login = Get-Login -Prenom "Jacques1" -Nom "OUZI"
```

Nous obtenons une erreur, car la valeur "Jacques1" ne respecte pas le modèle défini au sein de "ValidatePattern".

```
PS C:\Users\Utilisateur> $Login = Get-Login -Prenom "Jacques1" -Nom "OUZI"
Get-Login : Impossible de valider l'argument sur le paramètre «Prenom». L'argument «Jacques1» ne correspond pas
au modèle «^[a-z]+$». Indiquez un argument qui correspond à «^[a-z]+$» et réessayez.
Au caractère Ligne:1 : 28
+ $Login = Get-Login -Prenom "Jacques1" -Nom "OUZI"
+ ~~~~~
+ CategoryInfo          : InvalidData : (:) [Get-Login], ParameterBindingValidationException
+ FullyQualifiedErrorId : ParameterArgumentValidationError,Get-Login
```

Nous pourrions également utiliser "ValidatePattern" pour vérifier que la valeur correspond à une adresse e-mail sur un nom de domaine spécifique. Par exemple, la ligne ci-dessous permet de s'assurer que l'adresse e-mail contient bien "@sio.fr". :

```
param([ValidatePattern("@sio.fr$")]$Email)
```

Il est à noter que PowerShell prend en charge la possibilité de spécifier un message personnalisé à retourner à l'utilisateur lorsque la validation échoue. La syntaxe est la suivante :

```
[Parameter(Mandatory=$true)][ValidatePattern("^[a-z]+$",ErrorMessage="Valeur incorrecte")]$Nom
```

Dans l'exemple ci-dessus, si la valeur pour le paramètre Nom n'est pas acceptée, PowerShell retournera le texte "Valeur incorrecte".

ValidateSet

Le paramètre "ValidateSet" permet de définir un ensemble de valeurs acceptées. Il s'utilise de la façon suivante :

```
param([ValidateSet(1,2,3,10,20,30,100)]$Nombre)
```



Au sein des parenthèses, vous devez indiquer les valeurs autorisées. Si la valeur du paramètre "\$Nombre" est différente, alors la fonction ne s'exécutera pas.

ValidateLength

Le paramètre "**ValidateLength**" permet de définir une longueur minimale et maximale pour un paramètre. Il s'utilise de cette façon :

```
param([ValidateSet(1,10)]$Chaine)
```

Dans l'exemple ci-dessus, la valeur du paramètre "\$Chaine" doit être comprise entre 1 et 10 caractères, car le nombre de caractères correspond à sa longueur.

ValidateScript

Le paramètre "**ValidateScript**" permet d'exécuter un script de validation pour accepter ou non la valeur d'un paramètre. Il s'utilise de cette façon :

```
param([ValidateScript({Test-Path $_})]$Path)
```

Dans l'exemple ci-dessus, le paramètre "\$Path" est destiné à avoir pour valeur le chemin vers un répertoire. Grâce au script présent dans "**ValidateScript**", nous allons exécuter la commande "**Test-Path**" pour tester l'existence de ce chemin : cette commande retournera un booléen (vrai ou faux). Si le chemin est valide, la valeur sera acceptée et la fonction pourra s'exécuter.

ValidateNotNullOrEmpty

Le paramètre "**ValidateNotNullOrEmpty**" permet d'indiquer que ce paramètre n'accepte pas une valeur non nulle ou vide. Il s'utilise de cette façon :

```
param([ValidateNotNullOrEmpty()]$Chaine)
```

Dans l'exemple ci-dessus, la valeur du paramètre "\$Chaine" ne peut pas être vide ou nulle, sinon nous obtenons une erreur. Autrement dit, nous ne pouvons pas appeler le paramètre en indiquant :

```
Nom-Fonction -Chaine ""
```

Grâce à ces nouvelles notions, il est possible de mieux contrôler et configurer vos paramètres de configuration en PowerShell.



GERER LA POSITION DES PARAMETRES DE FONCTION

Dans une fonction PowerShell, et au même titre que c'est le cas avec certains cmdlets PowerShell, **vous avez la possibilité d'imposer la position d'un paramètre**. Autrement dit, vous pouvez imposer l'ordre selon lequel les paramètres doivent être indiqués au moment d'appeler une fonction, lorsque le nom des paramètres n'est pas spécifié. Nous allons étudier cette notion dans ce chapitre.

Pour indiquer la position d'un paramètre, vous devez spécifier un chiffre tout en sachant que la première position est "0". Cette déclaration doit être précisée comme ceci :

```
param([Parameter(Position=0)]$Nom)
```

Si vous souhaitez que ce paramètre soit obligatoire et que sa position soit "0", voici la syntaxe à adopter pour ajouter ces deux instructions sur un même paramètre :

```
param([Parameter(Mandatory=$true,Position=0)]$Nom)
```

Exemple

Nous allons reprendre l'exemple de la fonction "Get-Login" et associer la **position 0** au paramètre "**Prenom**" et la **position 1** au paramètre "**Nom**". Ce qui donne ceci :

```
function Get-Login {  
  
    param([Parameter(Mandatory=$true,Position=1)][ValidatePattern("^[a-z]+$")]$Nom,  
          [Parameter(Mandatory=$true,Position=0)][ValidatePattern("^[a-z]+$")]$Prenom)  
  
    $Identifiant = (($Prenom).Substring(0,1)).ToLower() + "." + ($Nom).ToLower()  
    return $Identifiant  
}
```

Après avoir ajouté ce paramétrage, sachez qu'il n'y aura pas de différence si vous précisez le nom du paramètre et sa valeur. Ainsi, ces deux commandes donneront le même résultat :

```
Get-Login -Prenom "Jacques" -Nom "OUZI"  
Get-Login -Nom "OUZI" -Prenom "Jacques"
```

La position du paramètre en jeu si nous appelons la fonction et que nous indiquons des valeurs sans préciser le nom des paramètres. Par exemple :

```
Get-Login "Jacques" "OUZI"
```

Ainsi, la fonction prendra la première valeur pour l'affecter au paramètre "**Prenom**" et la seconde valeur pour l'associer au paramètre "**Nom**". Le fait d'affecter un numéro de position à chaque paramètre permet de clarifier le comportement de la fonction lorsque nous l'appelons sans préciser les noms des paramètres.

